RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

CS1003
Programming in C

Module - 5

Content of Module-5

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Chapter-1 Structures and Unions

- Introduction to Structures and Unions
- Structure Definition and Declaration
- Accessing Structure Members: Accessing individual members of a structure using the dot (.) operator
- Array of Structures.
- Passing Structures to Functions: Structures can be passed as function arguments
- Allowing to work with and modify structure data within functions.
- Pointer to Structures. Accessing Structure Members through a Pointers:
- Access its members using the arrow operator (->).
- Passing Structures as Reference: Pass structures by reference to functions
- Allowing the function to modify the original structure.

Introduction to Structures and Unions

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- **Structures** are a way to group different types of variables under a single name.
  - These variables, known as members or fields, can be of different types.
  - Structures are useful when you need to represent a collection of related data.
  - To define structure use the struct Keyword
- **Unions** allow you to store different data types in the same memory location.
  - Unlike structures, where each member has its own memory, unions use the same memory location for all their members.
  - This means that at any one time, a union can hold only one of its members.

Syntax:

```
struct StructName {
    dataType member1;
    dataType member2;
    // more members
};
```

Syntax for Declaring a Structure Variable:

```
struct StructName variableName;
```

Syntax for Accessing Members:

```
variableName.member1;
```

Syntax:

```
union UnionName {
    dataType member1;
    dataType member2;
    // more members
};
```

Syntax for Declaring a Structure Variable:

```
union UnionName variableName;
```

Syntax for Accessing Members:

```
variableName.member1;
```

- **Structure** is the collection of variables of different datatypes (heterogeneous elements) under a single name.
- Structure is a user-defined data type in C which allows you to combine different data types to store in a particular type of record.
- For Ex. Student detail (Name, SRN n0, Mark, DOB, etc) Structure helps to construct a complex data type in more meaningful way.
- It is somewhat similar to an Array.
- The only difference is that array is used to store collection of similar data types while structure can store collection of heterogenous type of data.
- You can define a structure to hold this information.
-

- **Define and declare a structure variable:**
- To define a structure **struct** keyword is used.
- struct define a new data type which is a collection of heterogenous (different) type of data.
- **Declaring a Structure Variables:-**
- It is possible to declare variables of a structure, after the structure is defined.
- Structure variable declaration is similar to the declaration of variables of any other data types.

- **Example of Structure**

```
struct Book
{
char name[15];
int price;
int pages;
}b1,b2;
```

- Here the struct Book declares a structure to hold the details of book which consists of three data members, namely name, price and pages.
- These members are called structure data elements or members.
- Each member can have different data type, like name of char type and price and pages is of int type.
- Book is tag name. b1 and b2 are structure variable.

Accessing Structure Members: Accessing individual members of a structure using the dot (.) operator

- Accessing individual members of a structure in C is done using the dot (.) operator.
- This operator allows you to access and manipulate the fields of a structure variable.
- Accessing structure member
  - structName.memberName;
  - structName is the variable of the structure type.
  - memberName is the name of the member you want to access.

Structure Initialization:-

- Like any other data type, structure variable can also be initialized at compile time.

```
struct patient
{
float height;
int weight;
int age;
};
struct patient p1 = { 180.75 , 73, 23 };
        //initialization
```

(OR)

```
struct patient p1;
p1.height = 180.75;     //initialization of each
member separately
 p1.weight = 73;
p1.age = 23;
```

Accessing Structure Members:

- Structure members can be accessed and assigned values in number of ways.
- Structure member has no meaning independently.
- In order to assign a value to a structure member.
- The member name must be linked with the structure variable using **dot (.) operator** also called **period or member access operator.**
-

```
struct Book
{
char name[15];
 int price;
int pages;
} b1 , b2 ;
```

b1.price=200; //b1 is variable of Book type and price is member of Book

We can also use scanf() to give values to structure members through terminal.

scanf (" %s ", **b1.name**);

scanf(" %d ", &**b1.price**);

Program to declare a structure student with field name and roll no. take input from user and display them.

```
#include<stdio.h>
struct student
{
char name[10];
int roll;
};
void main()
{
struct student s1;

printf("\n Enter student record\n");
printf("\n student name\t");
scanf("%s",s1.name);
printf("\nEnter student roll\t");
scanf("%d",&s1.roll);
  printf(“\n Student detail is \n”);
  printf("\nstudent name is %s",s1.name);
   printf("\nroll is %d",s1.roll);
}
```

Program

```
#include <stdio.h>
#include <string.h>
// Define a structure
struct Person {
    char name[50];
    int age;
    float height;
};
int main() {
    // Declare a structure variable
    struct Person person1;

    // Assign values to the members
    strcpy(person1.name, "Alice");
    person1.age = 30;
    person1.height = 5.5;

    // Access and print the members
    printf("Name: %s\n", person1.name);
    printf("Age: %d\n", person1.age);
    printf("Height: %.1f\n", person1.height);
    return 0;
}
```

```
#include <stdio.h>
// Define a union
union Data {
    int intValue;
    float floatValue;
    char charValue;
};
int main() {
    // Declare a union variable
    union Data data;

    // Assign values to the members
    data.intValue = 10;
    printf("Int Value: %d\n", data.intValue);

    // Assign a value to another member (overwrites previous value)
    data.floatValue = 15.5;
    printf("Float Value: %.1f\n", data.floatValue);
    // Assign a value to another member (overwrites previous value)
    data.charValue = 'A';
    printf("Char Value: %c\n", data.charValue);
    return 0;
}
```

Difference between structure & Union

RV UNIVERSITY
Go, change the world
f RV EDUCATIONAL INSTITUTIONS

Feature	Structure	Union
Definition Syntax	`struct StructName { ... };`	`union UnionName { ... };`
Declaration Syntax	`struct StructName varName;`	`union UnionName varName;`
Memory Allocation	Allocates memory for all members.	Allocates memory for the largest member only.
Size	Total size is the sum of sizes of all members.	Size is the size of the largest member.
Access to Members	Each member has its own memory location.	All members share the same memory location.
Usage	Use when you need to store multiple related data items simultaneously.	Use when you need to store one of several possible data items, but only one at a time.
Initialization	Each member is initialized separately.	Only one member can be initialized at a time.

Summary of structure and Union

- **Memory Allocation**:
  - Structures allocate separate memory for each member,
  - While unions allocate memory sufficient for only the largest member, with all members sharing the same memory location.
- **Size**:
  - The size of a structure is the sum of the sizes of its members
  - Whereas the size of a union is the size of its largest member.
- · **Access**:
  - In structures, you can access and use multiple members simultaneously
  - But in unions, you can only use one member at a time because all members overlap in memory

Demonstration of Structure program

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

```
#include <stdio.h>
#include <string.h>

struct Person {
    char name[50];  // Array of 50 characters
    int age;       // Integer
    float height;   // Float
};
int main() {
    // Declare a structure variable
    struct Person person1;

    // Assign values to the members
    strcpy(person1.name, "Alice");
    person1.age = 30;
    person1.height = 5.5;

    // Print the size of each member and the total size of the structure
    printf("Size of name: %zu bytes\n", sizeof(person1.name)); // Typically 50 bytes
    printf("Size of age: %zu bytes\n", sizeof(person1.age));   // Typically 4 bytes
    printf("Size of height: %zu bytes\n", sizeof(person1.height)); // Typically 4 bytes
    printf("Total size of structure: %zu bytes\n", sizeof(person1)); // Total size including padding

    return 0;
}
```

Demonstration of Union program

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

```
#include <stdio.h>

union Data {
    int intValue;    // Integer
    float floatValue; // Float
    char charValue;  // Character
};

int main() {
    // Declare a union variable
    union Data data;

    // Assign values to different members
    data.intValue = 10;
    printf("Size of intValue: %zu bytes\n",
sizeof(data.intValue)); // Typically 4 bytes

    data.floatValue = 15.5;
    printf("Size of floatValue: %zu bytes\n",
sizeof(data.floatValue)); // Typically 4 bytes

    data.charValue = 'A';
    printf("Size of charValue: %zu bytes\n",
sizeof(data.charValue)); // Typically 1 byte

    // The size of the union is the size of the largest member
    printf("Total size of union: %zu bytes\n", sizeof(data)); // Size
of the largest member

    return 0;
}
```

Program to display student details

```
#include <stdio.h>
// Define a structure to hold student details
struct Date {
    int day;
    int month;
    int year;
};
struct Student {
    char name[50];
    int age;
    char section[10];
    struct Date dob;  // Date of Birth
};

int main() {
    struct Student student;
 printf("Enter student name: ");
    fgets(student.name, sizeof(student.name),
stdin);
printf("Enter student age: ");
    scanf("%d", &student.age);

    printf("Enter student section: ");
    scanf("%s", student.section);

    printf("Enter date of birth (day month year): ");
    scanf("%d %d %d", &student.dob.day,
&student.dob.month, &student.dob.year);

    // Print student details
    printf("\nStudent Details:\n");
    printf("Name: %s\n", student.name);
    printf("Age: %d\n", student.age);
    printf("Section: %s\n", student.section);
    printf("Date of Birth: %02d/%02d/%d\n", student.dob.day,
student.dob.month, student.dob.year);
    return 0;
}
```

Array of Structures

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- Structure is used to store the information of one particular object but if we need to store such 100 objects then Array of Structure is used.
- A structure is a collection of members of different data types stored in contiguous memory locations.
- An array of structures is an array in which each element is a structure.
- This concept is very helpful in representing multiple records of a file, where each record is a collection of dissimilar data items.
- As we have an array of integers, we can have an array of structures also.
- For example, suppose we want to store the information of class of students, consisting of name, roll_number and marks,

Syntax for array of structure

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

```
struct   tag_name
{
data type member1;
 data type member2;
data type member N;
} array_Var [size];
```

- Where
  - tag_name – Name given to a collection **ex** student, book, emp etc.
  - Data type – Primary data type .
  - Member1……N – Variable names for ex. Name, Per, Age, Address, etc.
  - array_var – Array of Structure variable.

Example for array of structure

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- **Example:**

```
struct Bookinfo
{
char bname;
int pages;
float price;
}b[3];
```

Explanation:

Here Book structure is used to Store the information of one Book.

In the above example if we need to store the Information of 3 books then Array of Structure is used.

b[0] stores the Information of 1st Book.

b[1] stores the information of 2nd Book and So on.

Accessing Pages field of Second Book -

b[1].pages

Example: Program for storing and displaying book detail using array of structure.

```
#include <stdio.h>
#include <string.h>
struct Bookinfo
{
char bname;
 int pages;
float price;
}b[3];
void main()
{
int i;
for(i=0;i<3;i++)
{
printf("\n Enter the Name of Book: ");
gets(b[i].bname);
printf("\nEnter the Number of Pages : ");
scanf("%d",b[i].pages);
printf("\nEnter the Price of Book: ");
scanf("%f",b[i].price);
}
printf("\n-------- Book Details");
for(i=0;i<3;i++)
{
printf("\nName of Book: %s", b[i].bname);
printf("\nNumber of Pages :%d", b[i].pages);
printf("\nPrice of Book: %f \n", b[i].price);
}
}
```

Output

```
--------- Book Details -----------
Name of Book: Programming with C
Number of Pages: 100
Price of Book: Rs. 200

Name of Book: C Programming
Number of Pages: 200
Price of Book: Rs.300

Name of Book: The Complete Reference C
Number of Pages: 300
 Price of Book: Rs.500
```

Program

```
#include <stdio.h>
 struct Employee
{
char ename[20];
 int ssn;
float salary;
 struct date
{
int date;
int month;
int year;
}doj;
}emp = {"Pritesh",1000,10000,{22,6,1990}};
void main()
{
printf("\n Employee Name: %s", emp.ename);
printf("\nEmployee SSN: %d", emp.ssn);
printf("\nEmployee Salary : %f", emp.salary);
printf("\nEmployee DOJ: %d/%d/%d", emp.doj.date, emp.doj.month, emp.doj.year);
}
```

Output

```
Employee Name: Pritesh
Employee SSN: 1000
Employee Salary : 10000.000000
Employee DOJ: 22/6/1990
```

Program to check size of union and structure

```
#include <stdio.h>
union job
{
char name[32];
float   salary;
int emp_no;
}u;
struct job1
{
char name[32];
float   salary;
int emp_no;
}s;
            int main()
{
printf("size of union = %d",sizeof(u));
printf("\nsize of structure = %d", sizeof(s));
return 0;
}
```

Output

size of union = 32
size of structure = 40

Chapter-2 File operations

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- Creating a new file
- opening an existing file,
- reading from file
- writing to a file
- moving to a specific location in a file
- Closing a file.

Introduction

- File operations in C are managed using the standard library functions provided in <stdio.h>.
- Here is a brief overview of basic file operations, including creating, opening, reading from, writing to, moving to a specific location in a file, and closing a file.

Basic File Operations in C

Creating a New File:

Use fopen with the mode "w" or "w+". If the file does not exist, it will be created. If it exists, it will be truncated.

Opening an Existing File:

Use fopen with modes such as "r" (read), "r+" (read and write), "a" (append), or "a+" (append and read).

Reading from a File:

Use functions like fgetc, fgets, or fread to read data from a file.

Writing to a File:

Use functions like fputc, fputs, or fwrite to write data to a file.

Moving to a Specific Location in a File:

Use fseek to move the file pointer to a specific location in the file.

Closing a File:

Use fclose to close an open file and release the resources.

Demonstration program with files operation

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

```
#include <stdio.h>

int main() {
   // File pointer declaration
   FILE *file;
   char buffer[100];
   // 1. Creating a new file and writing to it
   file = fopen("example.txt", "w"); // Create or truncate the file for writing
   if (file == NULL) {
      perror("Error opening file");
      return 1;
   }
   fprintf(file, "Hello, world!\nThis is a sample file.\n");
   fclose(file); // Close the file
   // 2. Opening an existing file and reading from it
   file = fopen("example.txt", "r"); // Open for reading
   if (file == NULL) {
      perror("Error opening file");
      return 1;
   }
   printf("File contents:\n");
   while (fgets(buffer, sizeof(buffer), file) != NULL) {
      printf("%s", buffer); // Read and print each line
   }
   fclose(file); // Close the file
// 3. Opening a file for updating (read and write)
   file = fopen("example.txt", "r+"); // Open for reading and writing
   if (file == NULL) {
      perror("Error opening file");
      return 1;
   }
}
```

```
// 4. Moving to a specific location and writing
    fseek(file, 0, SEEK_SET); // Move to the beginning of the file
    fprintf(file, "Updated content\n"); // Write at the beginning
    fclose(file); // Close the file
    // 5. Re-reading to verify changes
    file = fopen("example.txt", "r"); // Reopen for reading
    if (file == NULL) {
        perror("Error opening file");
        return 1;
    }
    printf("\nUpdated File contents:\n");
    while (fgets(buffer, sizeof(buffer), file) != NULL) {
        printf("%s", buffer); // Read and print each line
    }
    fclose(file); // Close the file
    return 0;
}
```

- Explanation of File Operations
- Creating and Writing to a File:
  - fopen("example.txt", "w") creates a new file or truncates an existing file.
  - fprintf(file, "Hello, world!\n") writes formatted data to the file.
  - fclose(file) closes the file after writing.
- Opening and Reading from a File:
  - fopen("example.txt", "r") opens an existing file for reading.
  - fgets(buffer, sizeof(buffer), file) reads a line from the file into buffer.
  - fclose(file) closes the file after reading.
- Moving to a Specific Location in a File:
  - fseek(file, 0, SEEK_SET) moves the file pointer to the beginning of the file.
  - fprintf(file, "Updated content\n") writes new data at the current position.
- Re-reading to Verify Changes:
  - Re-open the file with fopen("example.txt", "r") and read it again to verify that changes were made.

- Program to read content from file

```
#include <stdio.h>
int main() {
    // File pointer
    FILE *file;
    char buffer[256]; // Buffer to hold data read from the file
    // Open the file for reading
    file = fopen("example.txt", "r");
    if (file == NULL) {
        perror("Error opening file");
        return 1;
    }
// Read and print the file content line by line
    printf("File contents:\n");
    while (fgets(buffer, sizeof(buffer), file) != NULL) {
        printf("%s", buffer); // Print each line read from the file
    }
    // Close the file
    fclose(file);
    return 0;
}
```

- File Pointer:
  - FILE *file; is a pointer to the file structure used for file operations.
- Opening the File:
  - fopen("example.txt", "r") opens example.txt in read mode. If the file does not exist or fails to open, an error message is printed using perror, and the program exits with a status code of 1.
- Reading the File:
  - fgets(buffer, sizeof(buffer), file) reads a line from the file into buffer. The size of buffer limits the number of characters read in one call to fgets.
  - The while loop continues until fgets returns NULL, which indicates the end of the file or an error.
- Printing the File Content:
  - printf("%s", buffer) prints each line read from the file. fgets includes the newline character, so lines are printed correctly.
- Closing the File:
  - fclose(file) closes the file once all operations are complete.

- Program to write content from file

```
#include <stdio.h>
int main() {
    // File pointer
    FILE *file;
    // Open the file for writing
    file = fopen("output.txt", "w");
    if (file == NULL) {
        perror("Error opening file");
        return 1;
    }
    // Write some content to the file
    fprintf(file, "Hello, World!\n");
    fprintf(file, "This is a simple C program that writes to a file.\n");
    fprintf(file, "You can add more lines or data as needed.\n");
    // Close the file
    fclose(file);
    printf("Content successfully written to output.txt\n");
    return 0;
}
```

- File Pointer:
  - FILE *file; declares a pointer to a file structure used for file operations.
- Opening the File:
  - fopen("output.txt", "w") opens output.txt in write mode. If the file does not exist, it will be created. If it does exist, it will be truncated (i.e., existing content will be removed).
- Writing to the File:
  - fprintf(file, "Hello, World!\n"); writes formatted text to the file. You can use fprintf to format the output similarly to printf.
  - Each fprintf call writes a new line or piece of data to the file.
- Closing the File:
  - fclose(file) closes the file after writing. This ensures that all data is flushed from the buffer and saved to disk.
- Error Handling:
  - If the file cannot be opened (e.g., due to permissions issues), fopen returns NULL, and perror prints an error message.

Program to delete content from file

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

```
#include <stdio.h>
int main() {
    // File pointer
    FILE *file;
    // Open the file for writing
    file = fopen("example.txt", "w");
    if (file == NULL) {
        perror("Error opening file");
        return 1;
    }
    // At this point, the file is truncated and empty
    // Close the file
    fclose(file);
    printf("Content of 'example.txt' has been deleted.\n");
    return 0;
}
```

Explanation

- File Pointer:
  - FILE *file; declares a pointer to handle file operations.
- Opening the File for Writing:
  - fopen("example.txt", "w") opens example.txt in write mode. If the file exists, its content will be truncated (i.e., all previous content will be removed). If the file does not exist, it will be created.
- Closing the File:
  - fclose(file) closes the file. This is necessary to ensure all operations are completed and resources are released.
- Error Handling:
  - fopen returns NULL if the file cannot be opened (e.g., due to permission issues or invalid file paths). perror prints an error message in such cases.

Program to copy content from one file to another file

```
#include <stdio.h>
int main() {
    // File pointers
    FILE *sourceFile;
    FILE *destinationFile;
    char buffer[1024]; // Buffer to hold data
read from the source file
    size_t bytesRead;
    // Open the source file for reading
    sourceFile = fopen("source.txt", "r");
    if (sourceFile == NULL) {
        perror("Error opening source file");
        return 1;
    }
    // Open the destination file for writing (creates
it if it doesn't exist)
    destinationFile = fopen("destination.txt",
"w");
    if (destinationFile == NULL) {
        perror("Error opening destination file");
        fclose(sourceFile); // Close the source file
if destination file fails to open
        return 1;
    }
```

Cont...

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

```
// Read from the source file and write to the destination file
    while ((bytesRead = fread(buffer, 1, sizeof(buffer), sourceFile)) > 0) {
        fwrite(buffer, 1, bytesRead, destinationFile);
    }
// Close both files
    fclose(sourceFile);
    fclose(destinationFile);
    printf("Content successfully copied from source.txt to destination.txt\n");
    return 0;
}
```

Explanation

- File Pointers:
  - FILE *sourceFile; is used to read from the source file.
  - FILE *destinationFile; is used to write to the destination file.
- Opening the Files:
  - fopen("source.txt", "r") opens the source file in read mode. If the file cannot be opened, an error message is printed and the program exits.
  - fopen("destination.txt", "w") opens the destination file in write mode. If it does not exist, it will be created. If it cannot be opened, an error message is printed, and the source file is closed before the program exits.
- Reading and Writing Data:
  - fread(buffer, 1, sizeof(buffer), sourceFile) reads data from the source file into the buffer. sizeof(buffer) specifies the maximum number of bytes to read.
  - fwrite(buffer, 1, bytesRead, destinationFile) writes the data from the buffer to the destination file. bytesRead specifies the number of bytes read and written.
- Closing the Files:
  - fclose(sourceFile) closes the source file after reading.
  - fclose(destinationFile) closes the destination file after writing.


RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS
Thank You